DentaSync, Bright Smile Dental Clinic

Presentation cheat sheet: system specs, features and likely questions

One-line pitch: A full-stack web system that runs a 3-branch dental clinic in Makati. It handles scheduling with **database-guaranteed no double booking**, PDA dental charts, online booking, patient QR forms, billing with official receipts, supplies, and a privacy audit log (RA 10173).

1. Tech stack

Language	**TypeScript** (front end and back end)
Framework	**Next.js 16** (App Router), full stack
UI	**React 19**, **Tailwind CSS 4**, shadcn/ui, Lucide icons, light/dark mode
Client data	**TanStack Query 5** (caching, 30 s auto-refresh)
Validation	**Zod 4** on every input
Database	**PostgreSQL 17**. Dev/tests: **PGlite** (Postgres in WASM). Prod: **Supabase**, Singapore
ORM	**Drizzle ORM** + drizzle-kit migrations, plus hand-written SQL for constraints and triggers
Auth	**Better Auth 1.7**: username login, Postgres sessions, rate-limited sign-in
QR codes	`qrcode` package (SVG)
Testing	**Vitest** (~360 unit/DB/API tests) and **Playwright** (end-to-end)
Hosting	**Vercel** (Singapore, `sin1`) + Supabase

2. API

- Its own **REST API** at `/api/v1/...`, about 55 route handlers.
- Resources: appointments, availability, patients, chart entries, exams, notes, bills, billing close, branches, chairs, dentists, schedules, time off, staff, join requests, supplies, procedures, audit log, overview.
- Public portal: `/portal/bookings`, `/portal/times`, `/portal/forms`.
- Auth: Better Auth at `/api/auth/[...all]`.
- **No third-party APIs**: no SMS, email or payment gateway. QR payment is confirmed by hand by staff.
- One error shape: `{ error: { code, message, fields?, conflicts?, warnings? } }`
- Status codes: 400 invalid · 401 signed out · 403 forbidden · 404 not found · **409 clash** · 422 rule broken · 429 rate limited.

3. Roles

Role	Can do
Owner	Everything, all branches. Settings, access log. Charts if they see patients.
Manager / front desk	Book, confirm, check in, bill, close the day, approve staff (for their branches). Reads charts, never writes them.
Dentist	Own calendar and "My day", treat and complete own visits, write charts, exams and notes.
Patient	No login. Public booking page and patient form only.

4. Features

Accounts

- Staff sign up by **scanning a branch QR poster**; the owner or a manager approves.
- Password reset by one-time QR link (15 min, stored hashed). First-run `/setup`.

Scheduling

- Chair calendar (day/week), branch switcher, dentist "My day", all-branches overview.
- **No double booking** of dentist, chair or patient, including chair cleaning (turnover) time. Enforced by Postgres **exclusion constraints**.
- Cross-branch rule: a dentist scheduled at Branch A can't be booked at Branch B at the same time.
- Hard stops vs. warnings ("Book anyway" for overtime or outside clinic hours).
- Lifecycle: **Requested → Confirmed → Checked-In → In-Treatment → Completed**, plus No-Show and Cancelled. A **database trigger** enforces it.
- Dentist weekly schedules (split days across branches) and time off.

Clinical records

- **PDA dental chart**: FDI numbering, 5 surfaces per tooth, full PDA legend.
- Exam: periodontal, occlusion, appliances, TMD, X-rays. Treatment notes and visit history.
- Records are **never edited or deleted**, only voided with a reason.

Patient-facing

- Public landing page (services in English + Tagalog: Bunot, Pasta, Pustiso).
- **Online booking** `/book`: branch → service → time → details. The system assigns a dentist and chair; the front desk confirms.
- **Patient form via QR** `/welcome/{branch}`: the front desk turns it into a chart.
- Anti-spam: honeypot field, 5 per connection per hour, firewall rate limit.

Billing

- Checkout from a completed visit or as a walk-in sale. **Cash** (change computed) or **QR** (manual confirm).
- Gap-free receipt numbers `OR-BRANCH-YYYYMMDD-000001`, printable receipt.
- Void with a reason. **End-of-day reconcile** shows over/short; closed days lock.
- Money stored as integer **centavos**.

Supplies

- Per-branch stock with reorder levels. Every change is audited.

5. Security and privacy (RA 10173)

- **Audit/access log**: sign-ins, every patient record view, every change.
- One permission function `can(user, action, target)`, checked on the server for every request.
- Row-level security on every table. CSP with per-request nonce, HSTS, frame-deny, nosniff.
- Hashed passwords, 12-hour httpOnly sessions, cross-origin writes refused.
- Data stays in Singapore. Nightly **GPG-encrypted** backups. Patient details never written to logs.

6. Demo flow

Book visit → Check in → Start treatment → Chart a tooth → Exam + note → Complete → Bill (cash) → Print receipt

Then: **try a double booking** to show the 409 refusal. Show `/book` on a phone, and the access log in Settings.

7. Likely questions and suggested answers

General

1. What problem does it solve?

Paper/spreadsheet scheduling across 3 branches causes double bookings and lost records. This puts scheduling, charts, billing and supplies in one place.

2. Who are the users?

Owner, managers/front desk, dentists, and patients (public pages only, no login).

3. What's out of scope?

SMS/email, payment gateway, patient accounts, analytics, VAT/discounts, full lot-based inventory. These are planned future parts.

Architecture and technology

4. Why Next.js instead of a separate front end and back end?

One codebase and one language (TypeScript). Server pages and the REST API call the same service functions.

5. Why PostgreSQL, not MySQL or MongoDB?

Exclusion constraints and triggers let the **database** guarantee no overlaps. The data is highly relational.

6. What is PGlite?

Postgres running inside Node (WASM). Dev and tests need no installed database and behave exactly like production.

7. What is an ORM, and why Drizzle?

It maps tables to code. Drizzle gives type-safe queries, and migrations are plain SQL files.

8. Is it REST? Show an endpoint.

`POST /api/v1/appointments` books a visit; `/appointments/validate` checks a booking before saving.

9. Hosting and cost?

Vercel + Supabase in Singapore. Supabase Free; Vercel Pro (~$20/month) because Hobby doesn't allow commercial use.

Core logic

10. How do you prevent double booking, even with two front desks at once?

Postgres exclusion constraints. The second save fails with 409: "Dr. X was just booked at 10:00."

11. What is the turnover buffer?

Cleaning time after a visit. It blocks the **chair**, not the dentist.

12. How are open times calculated?

15-minute grid: dentist scheduled at that branch, not on time off, a free chair including turnover, no clash for the patient, start not in the past.

13. Can a completed visit be cancelled?

No. Completed and Cancelled are final, and a database trigger enforces the allowed transitions.

14. How do you handle time zones?

Stored as `timestamptz`; every day boundary and display uses Asia/Manila.

15. How does online booking pick a dentist?

From the dentists allowed for that service who are free; the lowest-numbered free chair is picked. Staff confirm it.

Security

16. Authentication and passwords?

Better Auth hashes passwords. 12-hour httpOnly sessions, rate-limited sign-in.

17. How do you control what each role can do?

One `can()` permission function, checked on the server for every request and unit-tested for every role.

18. Data Privacy Act compliance?

Audit log, privacy notice + consent, data in one region, encrypted backups, no patient data in logs.

19. What stops spam bookings?

Honeypot field, limits per connection (hashed IP) and per patient, Vercel firewall rate limit.

20. Why can't records be edited or deleted?

Medical and financial records need integrity, so mistakes are voided with a reason and stay visible.

Billing

21. Why integer centavos?

Floating-point numbers cause rounding errors with money.

22. How are receipt numbers gap-free and unique?

A counter per branch per day is incremented inside the same transaction that issues the bill.

23. Why is QR payment confirmed by hand?

No gateway fees or integration. Staff check their own e-wallet app, then press "Payment received".

Testing and future work

24. How did you test it?

~360 Vitest tests (unit, database, API, each role's permissions) + a Playwright end-to-end run of a full visit.

25. What if data is lost?

Nightly encrypted `pg_dump` backups, with a restore tested before go-live.

26. What would you add next?

SMS/email reminders, patient accounts, analytics, full inventory with expiry dates, payment gateway, two-factor sign-in.

27. Hardest part?

The conflict and cross-branch scheduling rules, or the PDA chart's surface logic. Pick the one you worked on.

28. Can it scale to more branches?

Yes. Branches, chairs and dentists are data, not code, and Postgres handles the concurrency.